Web Servers

By : Muhammed Gamal

Agenda

Table of Contents

1. Introduction to Nginx
2. Installation and Setup
3. Basic Configuration
4. Serving Static Content
5. Virtual Hosts (Server Blocks)
6. Reverse Proxy Fundamentals
7. Load Balancing
8. Basic Security
9. Troubleshooting and Monitoring

What is Nginx?

Definition

- **Nginx** (pronounced "engine-x") is a high-performance web server and reverse proxy server
- Originally developed by Igor Sysoev in 2004
- Open-source software written in C

High Performance: Can handle thousands of concurrent connections

Low Memory Usage: Very efficient with system resources

Reverse Proxy: Can forward requests to backend servers

Load Balancer: Distribute traffic across multiple servers

Static Content Serving: Excellent for serving files, images, CSS, JS

SSL/TLS Termination: Handle HTTPS encryption/decryption

What is Nginx?

Why Use Nginx?

- **Speed**: Faster than Apache for static content
- **Scalability**: Event-driven architecture handles many connections efficiently
- **Flexibility**: Can act as web server, reverse proxy, load balancer
- **Reliability**: Powers many high-traffic websites (Netflix, Airbnb, WordPress.com)

Lab 1: Environment Setup and Basic Exploration

Task 1.1: Check if Nginx is Installed

Task 1.2: Install Nginx (if needed)

Task 1.3: Basic Nginx Commands

Task 1.4: Test Basic Installation

Nginx Installation and Setup

Installation Methods

1. **Package Manager** (Recommended for beginners)
   - Ubuntu/Debian: `apt install nginx`
   - CentOS/RHEL: `yum install nginx`
   - Fedora: `dnf install nginx`
2. **Official Repository** (For latest version)
   - Provides more recent versions
   - Better security updates
3. **Source Compilation** (Advanced users)
   - Custom modules and optimizations

Nginx Installation and Setup

Directory Structure :

```
/etc/nginx/                    # Main configuration directory
├── nginx.conf                 # Main configuration file
├── sites-available/           # Available server configurations
├── sites-enabled/             # Active server configurations (symlinks)
├── conf.d/                    # Additional configuration files
└── modules-enabled/           # Enabled modules

/var/log/nginx/                # Log files
├── access.log                 # Access logs
└── error.log                  # Error logs

/var/www/html/                 # Default document root
/usr/share/nginx/html/         # Alternative document root (CentOS)
```

Lab 2: Installation and File System Exploration

Task 2.1: Verify Installation Directories

Task 2.2: Examine Default Configuration

Task 2.3: Check Log Files

Task 2.4: Locate Document Root

Basic Nginx Configuration

Configuration Structure

Nginx configuration uses a hierarchical structure with contexts:

```
# Main context
user nginx;
worker_processes auto;

# Events context
events {
    worker_connections 1024;
}

# HTTP context
http {
    # Server context
    server {
        listen 80;
        server_name example.com;

        # Location context
        location / {
            root /var/www/html;
            index index.html;
        }
    }
}
```

Key Directives

- **listen**: Port and IP to listen on
- **server_name**: Domain names this server block handles
- **root**: Document root directory
- **index**: Default files to serve
- **location**: Handle specific URL patterns

Configuration Syntax

- Directives end with semicolons (`;`)
- Blocks are enclosed in curly braces (`{}`)
- Comments start with hash (#)
- Include files with `include` directive

Lab 3: Basic Configuration Modification

Task 3.1: Create a Custom HTML Page

Task 3.2: Test Configuration Syntax

Task 3.3: View Your Custom Page

Task 3.4: Modify Default Port

Serving Static Content

Static Content Types

- **HTML files**: Web pages
- **CSS files**: Stylesheets
- **JavaScript files**: Client-side scripts
- **Images**: JPG, PNG, GIF, SVG
- **Documents**: PDF, DOC, etc.
- **Media files**: Videos, audio files

Serving Static Content

Location blocks define how nginx handles different URL patterns:

```
server {
    listen 80;
    server_name example.com;
    root /var/www/html;

    # Exact match
    location = /favicon.ico {
        log_not_found off;
        access_log off;
    }

    # Static assets
    location ~* \.(css|js|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
        add_header Cache-Control "public";
    }

    # Default location
    location / {
        try_files $uri $uri/ =404;
    }
}
```

Performance Optimization

- **Gzip compression**: Reduce file sizes
- **Browser caching**: Set appropriate headers
- **File serving**: Optimize for static content delivery

Lab 4: Serving Static Content and Assets

Task 4.1: Create Website Structure

Task 4.2: Create JavaScript File

Task 4.3: Create Main HTML Page

Task 4.4: Configure nginx for the New Site

Task 4.5: Enable the Site

Virtual Hosts (Server Blocks)

What are Virtual Hosts?

Virtual hosts allow one nginx server to host multiple websites with different domain names.

Types of Virtual Hosts

1. **Name-based**: Multiple domains on same IP

2. **IP-based**: Different IPs for different sites

3. **Port-based**: Different ports for different sites

Virtual Hosts (Server Blocks)

Server Block Structure

```
# Site 1
server {
    listen 80;
    server_name site1.com www.site1.com;
    root /var/www/site1;
    index index.html;
}

# Site 2
server {
    listen 80;
    server_name site2.com www.site2.com;
    root /var/www/site2;
    index index.html;
}

# Default server (catch-all)
server {
    listen 80 default_server;
    server_name _;
    return 444;  # Close connection
}
```

Best Practices

- Use separate files for each site
- Keep configurations in `sites-available/`
- Symlink active sites to `sites-enabled/`
- Always include a default server block

Lab 5: Creating Multiple Virtual Hosts

Task 5.1: Create Multiple Website Directories

Task 5.2: Create Content for Company Site

Task 5.3: Create Content for Blog Site

Task 5.4: Create Content for Portfolio Site

Task 5.5: Create Server Block Configurations

Task 5.6: Enable All Sites

Task 5.7: Add Domain Names to Hosts File

Task 5.8: Test All Virtual Hosts

Slide 6: Reverse Proxy Fundamentals

What is a Reverse Proxy?

A reverse proxy sits between clients and backend servers, forwarding client requests to backend servers and returning responses back to clients.

Reverse Proxy vs Forward Proxy

- **Forward Proxy**: Client → Proxy → Internet
- **Reverse Proxy**: Client → Proxy → Backend Server

Common Use Cases

1. **Load Distribution**: Spread requests across multiple servers
2. **SSL Termination**: Handle HTTPS encryption/decryption
3. **Caching**: Store frequently requested content
4. **Compression**: Reduce bandwidth usage
5. **Security**: Hide backend server details

Slide 6: Reverse Proxy Fundamentals

Basic Reverse Proxy Configuration

```
server {
    listen 80;
    server_name api.example.com;

    location / {
        proxy_pass http://backend-server:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Key Proxy Directives

- **proxy_pass**: Backend server URL
- **proxy_set_header**: Set headers for backend
- **proxy_cache**: Enable caching
- **proxy_timeout**: Connection timeouts

Lab 6: Setting Up Reverse Proxy

Task 6.1: Create a Simple Backend Server

Task 6.2: Start the Backend Server

Task 6.3: Configure Nginx Reverse Proxy

Task 6.4: Enable and Test the Proxy

Task 6.5: Advanced Proxy Configuration

Task 6.6: Test Advanced Features

Load Balancing

What is Load Balancing?

Load balancing distributes incoming requests across multiple backend servers to ensure no single server becomes overwhelmed.

Load Balancing Methods

1. **Round Robin** (default): Requests distributed evenly
2. **Least Connections**: Send to server with fewest active connections
3. **IP Hash**: Route based on client IP
4. **Weighted**: Assign different weights to servers

Load Balancing

Load Balancing Configuration

```
upstream backend_pool {
    server backend1.example.com:8000 weight=3;
    server backend2.example.com:8000 weight=2;
    server backend3.example.com:8000 weight=1;
    server backup.example.com:8000 backup;
}

server {
    listen 80;
    server_name app.example.com;

    location / {
        proxy_pass http://backend_pool;
    }
}
```

Advanced Features

- **Health Checks**: Monitor server availability
- **Session Persistence**: Stick sessions to specific servers
- **Failover**: Automatic switching to backup servers
- **Dynamic Configuration**: Add/remove servers without restart

Lab 7: Load Balancing Setup

Task 7.1: Create Multiple Backend Servers

Task 7.2: Start All Backend Servers

Task 7.3: Configure Round Robin Load Balancing

Task 7.4: Test Round Robin Load Balancing

Task 7.5: Configure Weighted Load Balancing

Task 7.6: Configure Least Connections Load Balancing

Task 7.7: Test All Load Balancing Methods

Task 7.8: Test Failover

Basic Security

Security Fundamentals

Security is crucial for any web server deployment. Nginx provides several built-in security features.

Common Security Measures

1. **Hide Server Information**: Don't reveal nginx version
2. **Access Control**: Restrict access by IP, user agent, etc.
3. **Rate Limiting**: Prevent abuse and DDoS attacks
4. **SSL/TLS**: Encrypt communication
5. **Security Headers**: Protect against common attacks

Basic Security

Basic Security Configuration

```
# Hide nginx version
server_tokens off;

# Security headers
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "no-referrer-when-downgrade" always;

# Rate limiting
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req zone=api burst=20 nodelay;
```

Basic Security

Access Control Examples

```
# Allow specific IPs only
location /admin {
    allow 192.168.1.0/24;
    allow 10.0.0.0/8;
    deny all;
}

# Deny bad user agents
if ($http_user_agent ~* (nmap|wget|curl)) {
    return 403;
}
```

Lab 8: Implementing Basic Security

Task 8.1: Configure Security Headers

Task 8.2: Configure Rate Limiting in Main Config

Task 8.3: Create Secure Website Content

Task 8.4: Enable and Test Security Configuration

Task 8.5: Test Security Headers

Task 8.6: Test IP Access Control

Task 8.7: Test Rate Limiting

Task 8.8: Create Security Monitoring Script

Troubleshooting and Monitoring

Common nginx Issues

1. **Configuration Errors**: Syntax mistakes, missing semicolons
2. **Permission Problems**: File access issues
3. **Port Conflicts**: Multiple services on same port
4. **Resource Exhaustion**: High CPU/memory usage
5. **Backend Connectivity**: Upstream server problems

Troubleshooting and Monitoring

Essential Commands for Troubleshooting

```
# Test configuration syntax
nginx -t

# Reload configuration
systemctl reload nginx

# Check service status
systemctl status nginx

# View error logs
tail -f /var/log/nginx/error.log

# Check listening ports
netstat -tlnp | grep nginx
ss -tlnp | grep nginx
```

Troubleshooting and Monitoring

Log Analysis

```
# Monitor access logs in real-time
tail -f /var/log/nginx/access.log

# Find 404 errors
grep "404" /var/log/nginx/access.log

# Find slow requests
awk '$10 > 5' /var/log/nginx/access.log

# Top IPs by request count
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr
```

Performance Monitoring

- **Response times**: Monitor request processing time
- **Error rates**: Track 4xx and 5xx responses
- **Connection counts**: Monitor active connections
- **Resource usage**: CPU, memory, disk I/O

Lab 9: Troubleshooting and Monitoring

Task 9.1: Create Intentional Configuration Errors

Task 9.2: Practice Troubleshooting

Task 9.3: Create Monitoring Scripts

Task 9.4: Create Log Analysis Script

Task 9.5: Create Health Check Script

Task 9.6: Set Up Automated Monitoring

Task 9.7: Test Monitoring and Alerting

Task 9.8: Create Troubleshooting Playbook

Common Issues and Solutions

Issue: Configuration test fails

Symptoms: `nginx -t` shows syntax errors **Solution:**

1. Check for missing semicolons
2. Verify brackets are balanced
3. Check file permissions on included files

Issue: Permission denied errors

Symptoms: 403 errors in access log **Solution:**

1. Check file permissions: `ls -la /var/www/`
2. Verify nginx user: `ps aux | grep nginx`
3. Fix permissions: `sudo chown -R www-data:www-data /var/www/`

Common Issues and Solutions

Issue: Cannot bind to port

Symptoms: nginx fails to start, "Address already in use" **Solution:**

1. Check what's using the port: `ss -tlnp | grep :80`
2. Kill conflicting process or change nginx port

Issue: Upstream server errors

Symptoms: 502 Bad Gateway errors **Solution:**

1. Check if backend service is running
2. Verify backend connectivity: `curl http://backend:port`
3. Check proxy_pass configuration

Common Issues and Solutions

Issue: High memory usage

Symptoms: Server becomes slow **Solution:**

1. Check worker processes: `grep worker_processes /etc/nginx/nginx.conf`
2. Monitor connections: `ss -s`
3. Adjust worker_connections if needed

Emergency Procedures

Nginx won't start

1. `sudo nginx -t` (check config)
2. `sudo systemctl status nginx -l` (detailed status)
3. `sudo journalctl -u nginx` (system logs)
4. Check disk space: `df -h`

Common Issues and Solutions

High load emergency

1. Check active connections: `ss -s`
2. Implement rate limiting
3. Scale backend services
4. Consider enabling caching

Security incident

1. Check access logs for suspicious activity
2. Block malicious IPs: `deny IP_ADDRESS;`
3. Review security headers
4. Check for DDoS patterns

Final Lab: Complete nginx Setup Project

Project Objectives

Create a complete, production-ready nginx setup that incorporates all concepts learned.

Project Requirements

1. **Multiple websites** with different purposes
2. **Load-balanced backend** services
3. **Security configurations** implemented
4. **Monitoring and logging** set up
5. **SSL/TLS configuration** (bonus)

Final Lab: Complete nginx Setup Project

Project Tasks

Task 1: Architecture Design

Design a complete nginx setup with:

- Main corporate website
- API gateway
- Admin interface
- Load-balanced application servers
- Monitoring dashboard

Task 2: Implementation

Implement the complete solution using all techniques learned.

Final Lab: Complete nginx Setup Project

Task 3: Testing and Validation

- Load testing
- Security testing
- Failover testing
- Performance monitoring

Task 4: Documentation

Create complete documentation including:

- Architecture diagrams
- Configuration explanations
- Troubleshooting guides
- Maintenance procedures

Summary and Next Steps

What You've Learned

1. **nginx Fundamentals**: Installation, configuration, basic concepts
2. **Static Content Serving**: Optimized delivery of web assets
3. **Virtual Hosts**: Multiple websites on one server
4. **Reverse Proxy**: Backend service integration
5. **Load Balancing**: High availability and scalability
6. **Security**: Basic protection and hardening
7. **Troubleshooting**: Monitoring, debugging, and maintenance

Advanced Topics to Explore

- **SSL/TLS Configuration**: HTTPS setup and certificate management
- **Caching**: Proxy caching and FastCGI caching
- **Modules**: Third-party modules and custom configurations
- **Container Deployment**: nginx in Docker and Kubernetes
- **Advanced Security**: WAF, DDoS protection, advanced monitoring
- **Performance Tuning**: Optimization for high-traffic scenarios

Best Practices Recap

1. **Always test configurations** before reloading
2. **Keep configurations organized** and well-documented
3. **Monitor logs regularly** for issues and security events
4. **Implement proper security** from the beginning
5. **Plan for scalability** and high availability
6. **Backup configurations** and have rollback procedures
7. **Stay updated** with nginx releases and security patches

Resources for Continued Learning

- Official nginx documentation
- nginx community forums
- Security best practices guides
- Performance optimization resources
- Real-world case studies and examples